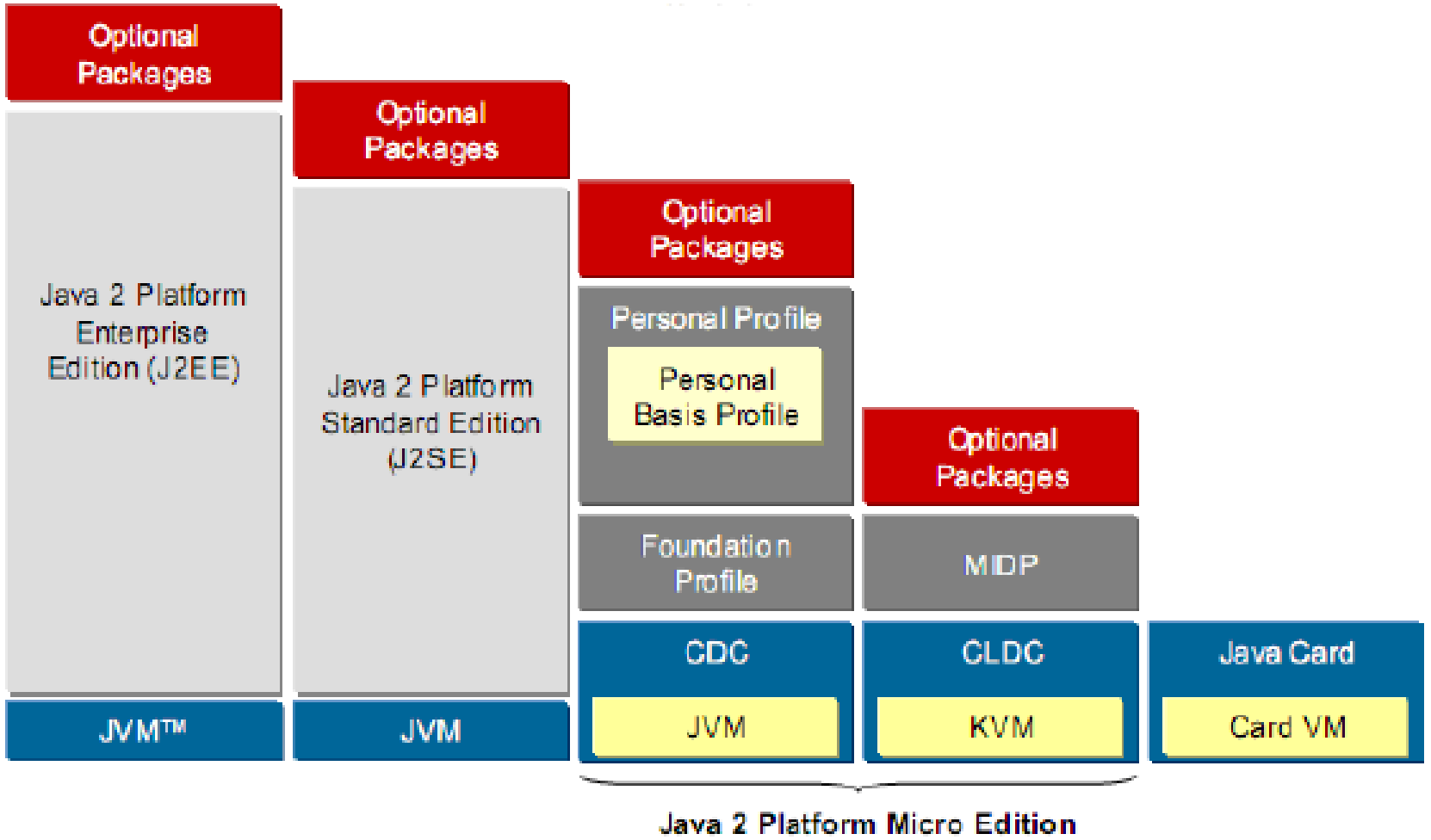
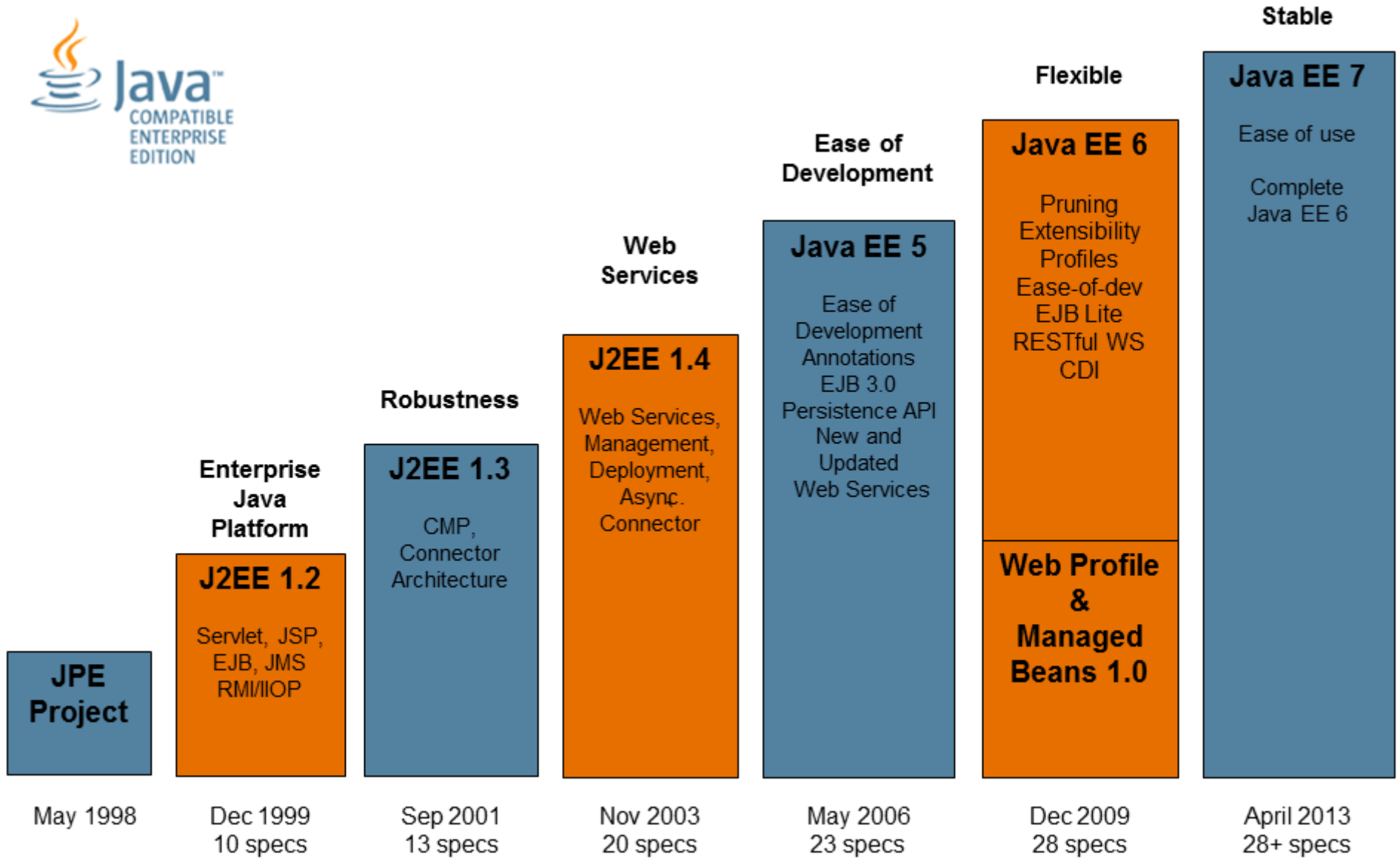
Business Components Development I

J2EE

Introduction to J2EE

J2EE, or Java 2 Platform, Enterprise Edition, is a set of specifications, APIs, and protocols for building scalable, reliable, and secure enterprise applications. It provides a comprehensive platform for developing and deploying multi-tiered, distributed, and web-enabled applications.





Importance in Enterprise Application Development to J2EE

In today's digital age, businesses require robust and flexible solutions to meet their complex requirements. J2EE offers a standardized and efficient framework for developing such applications, making it a cornerstone of enterprise software development.

Key Concepts:

J2EE Architecture

- Presentation Tier
- Business Logic Tier
- Data Tier

Components

- J2EE defines various components that operate within this architecture, including Servlets, JSP, EJB, JMS, JDBC, etc.

Container Services

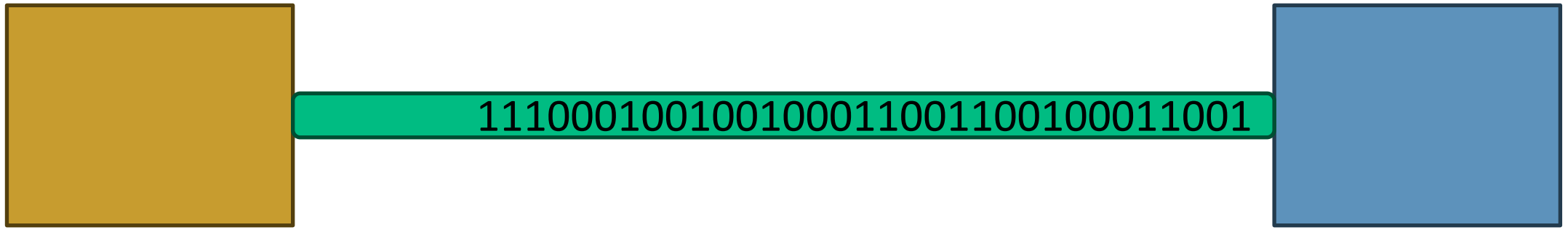
- J2EE containers provide essential services to components, such as transaction management, security, naming, and resource management. These services abstract the complexities of low-level programming tasks, allowing developers to focus on business logic.

What is cloud computing?

Hello

Hello

11100010010010001100110010001100 1



X – 3306
Y – 8080
Z - 1162

A

B

0 - 65535

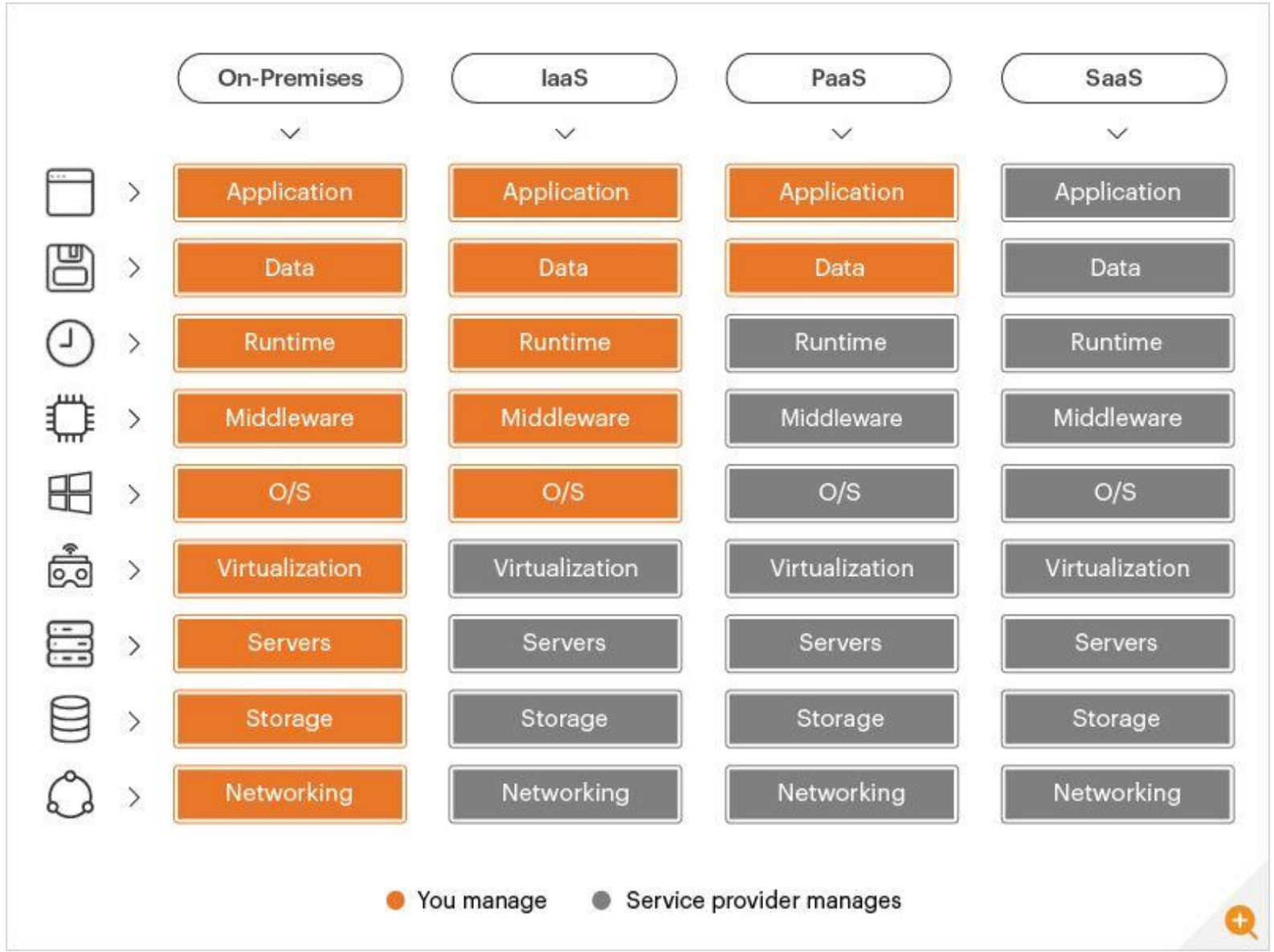
Simply put, cloud computing is the delivery of computing services—including servers, storage, databases, networking, software, analytics, and intelligence—over the Internet (“the cloud”) to offer faster innovation, flexible resources, and economies of scale. You typically pay only for cloud services you use, helping you lower your operating costs, run your infrastructure more efficiently, and scale as your business needs change.

Types of cloud services: IaaS, PaaS, and SaaS

Infrastructure as a service (IaaS)
The most basic category of cloud computing services. With IaaS, you rent IT infrastructure—servers and virtual machines (VMs), storage, networks, operating systems—from a cloud provider on a pay-as-you-go basis.

Platform as a service (PaaS)
Platform as a service refers to cloud computing services that supply an on-demand environment for developing, testing, delivering, and managing software applications. PaaS is designed to make it easier for developers to quickly create web or mobile apps, without worrying about setting up or managing the underlying infrastructure of servers, storage, network, and databases needed for development.

Software as a service (SaaS)
Software as a service is a method for delivering software applications over the Internet, on demand and typically on a subscription basis. With SaaS, cloud providers host and manage the software application and underlying infrastructure, and handle any maintenance, like software upgrades and security patching. Users connect to the application over the Internet, usually with a web browser on their phone, tablet, or PC.



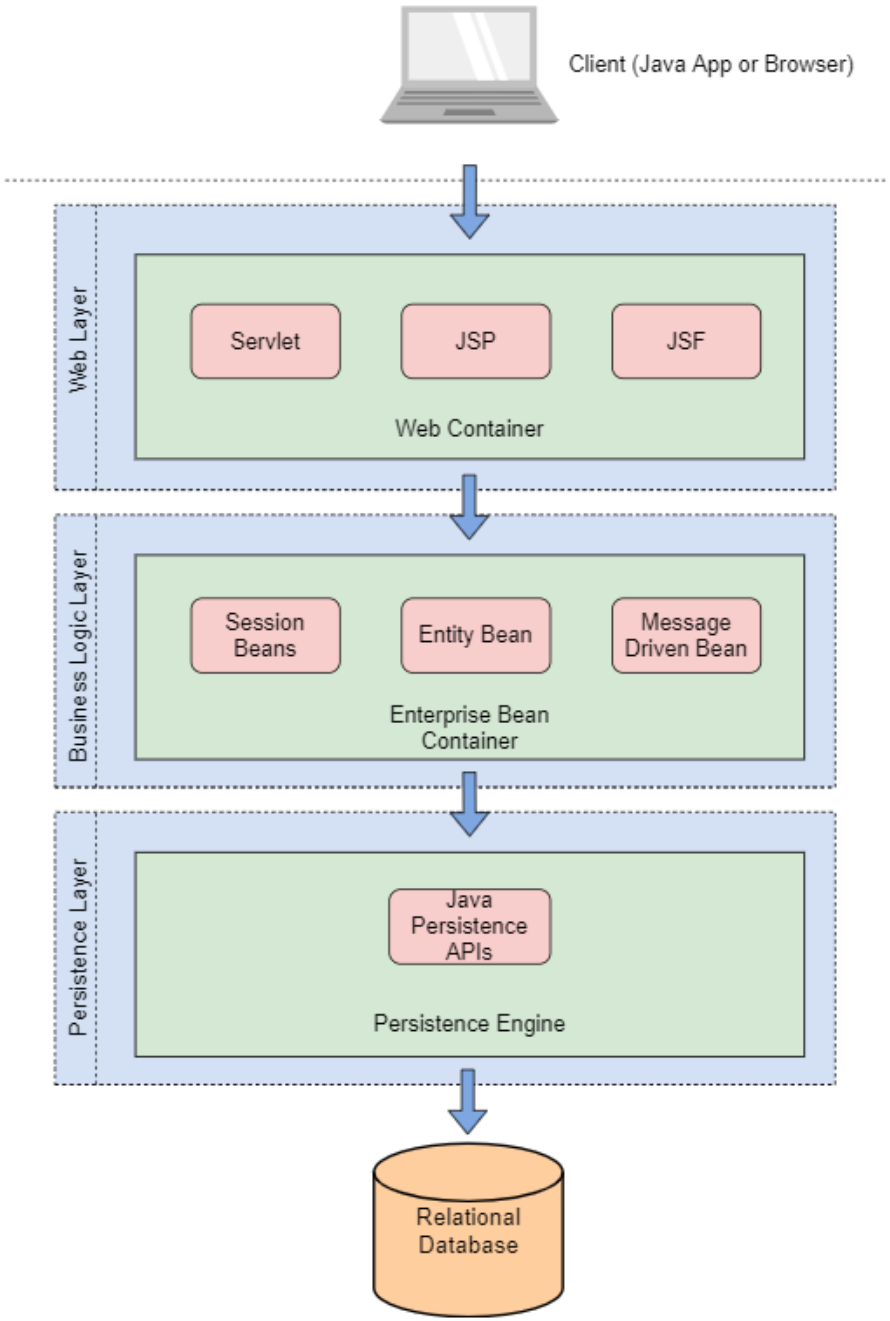
J2EE stands for Java 2 Platform, Enterprise Edition.

J2EE is the standard platform for developing applications in the enterprise and is designed for enterprise applications that run on servers.

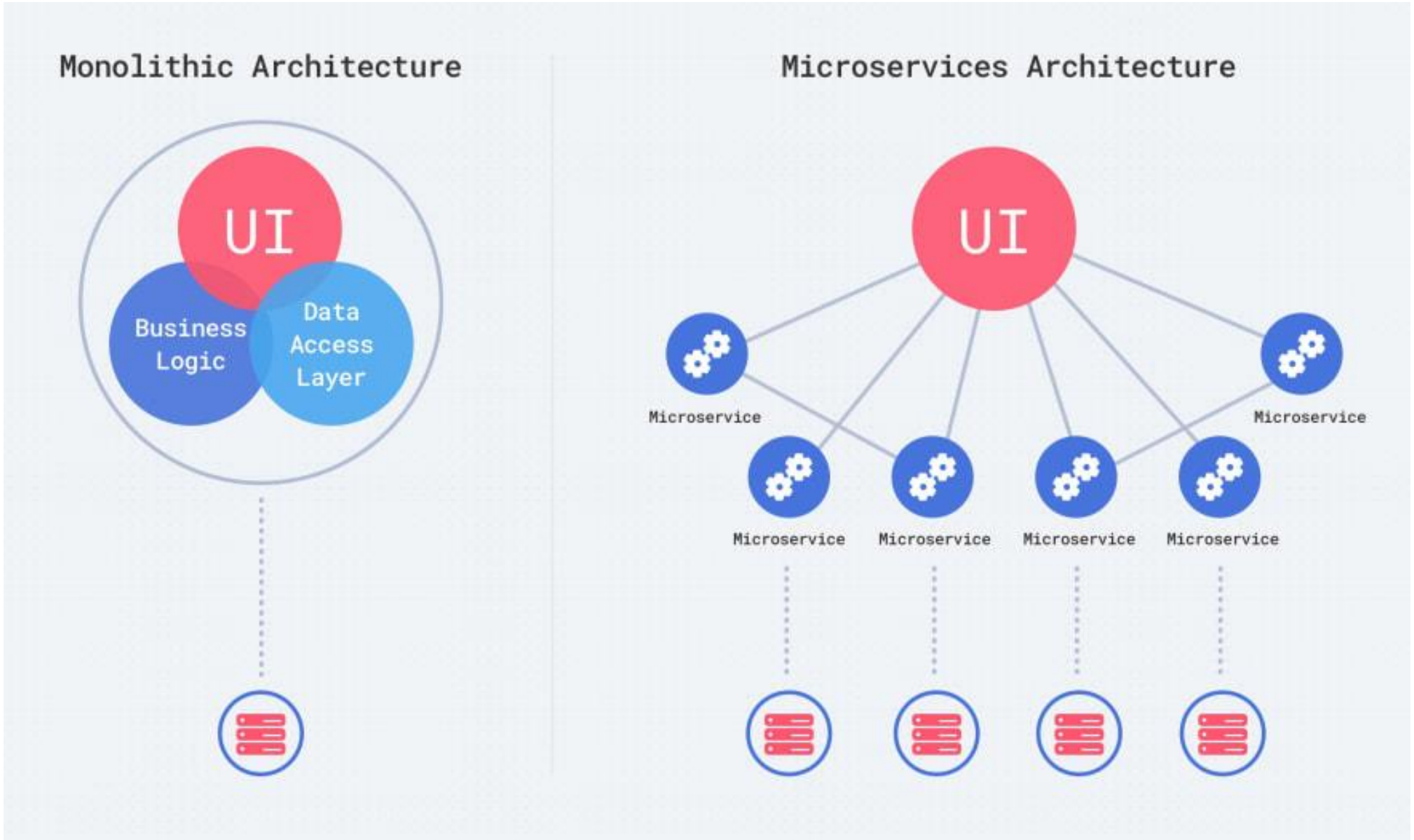
J2EE provides APIs that let developers create workflows and make use of resources such as databases or web services.

J2EE consists of a set of APIs. Developers can use these APIs to build applications for business computing.

A J2EE application server is software that runs applications built with J2EE APIs and lets you run multiple applications on a single computer. Developers can use different J2EE application servers to run applications built with J2EE APIs.



Multi-tier Architecture of Enterprise Java



Benefits of J2EE

1.Portability: If you build a J2EE application on a specific platform, it runs the same way on any other J2EE-compliant platform. This makes it easy to move applications from one environment to another. For example, moving an application from one computer to another or relocating an application from a test server to a production server.

2.Reusability: The components in J2EE are reused, so the average size of an application is much smaller than it would be if you had to write equivalent functionality from scratch for each program. For example, one component lets you read objects from a database. You can use that object-reading feature in any J2EE application. Since this functionality is already written and tested, you don’t have to write it yourself every time you need it.

3.Security: Java technology lets programmers handle sensitive data far more securely than they can in C/C++ programs.

4.Scalability: J2EE lets developers build applications that run well on both small, single-processor computers and large, multi-processor systems.

5.Reliability: Many of the services (such as transaction management and monitoring) that applications need to be reliable are built into J2EE.

Java Build Tools

What is Builder Tool ?

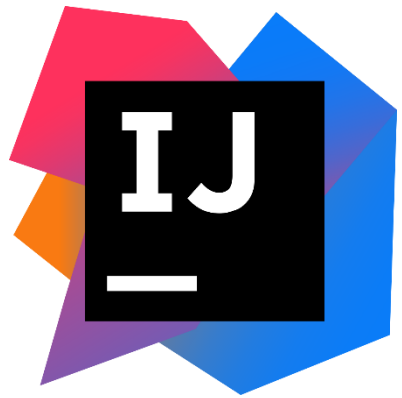
Build automation is the act of scripting or automating a wide variety of task that Software developers do in their day-to-day activities including things like

- Compiling computer source code into binary code
- Packaging binary code
- Running automated tests
- Deploying to production systems
- Creating documentation and/or release notes

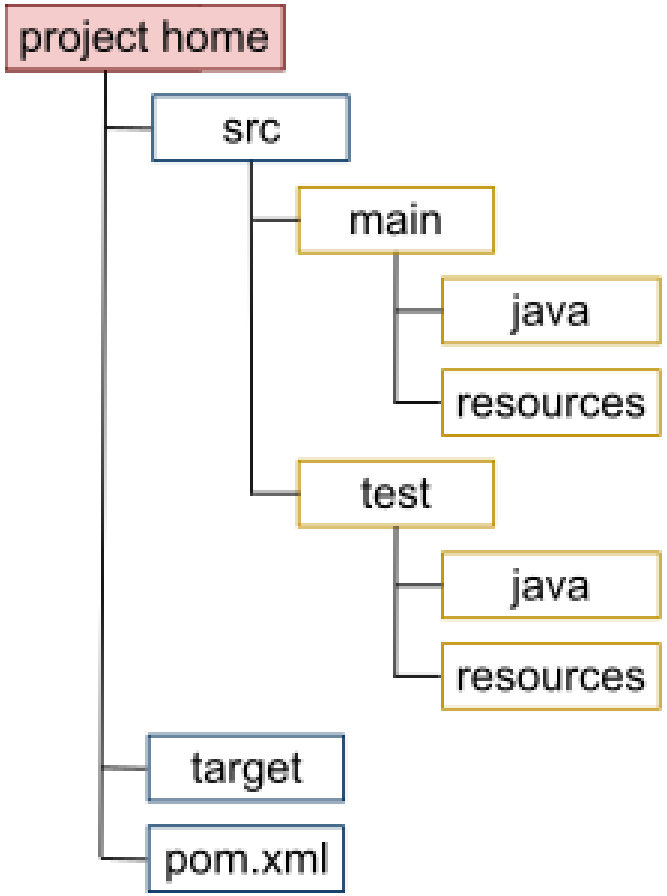
Maven™

gradle

eclipse



Structure of the Maven project



CORBA

COMMON OBJECT REQUEST BROKER ARCHITECTURE

CORBA, or **Common Object Request Broker Architecture**, is a standard defined by the Object Management Group (OMG) that enables software components written in different programming languages and running on different platforms to communicate with each other.

Key Features of CORBA:

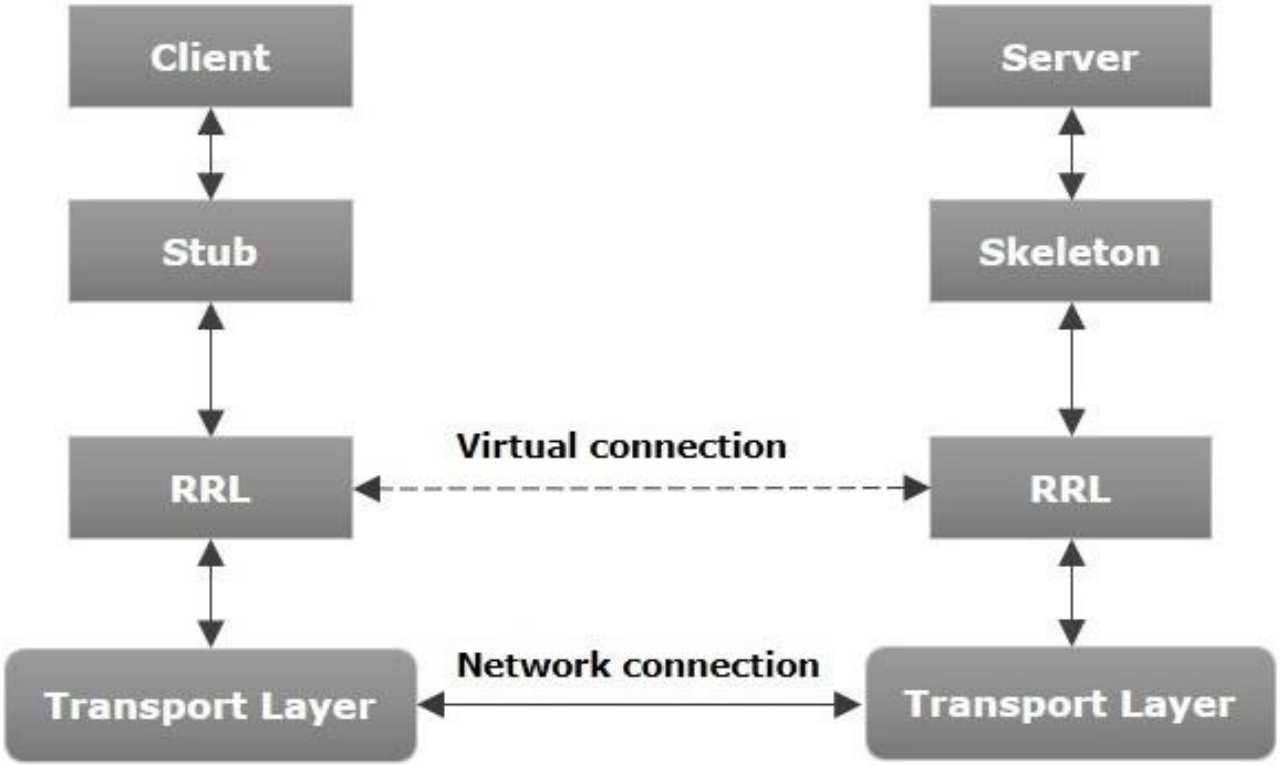
- **Language-independent (Java, C++, Python, etc.)**
- **Platform-independent**
- **Interoperability**: Allows different applications to work together, regardless of the language or platform they are built on.
- **Object-Oriented**: Supports the object-oriented programming paradigm, allowing developers to create reusable software components.
- **Remote Method Invocation**: Enables objects to invoke methods on other objects located remotely, as if they were local.
- **IDL (Interface Definition Language)**: Uses IDL to define the interfaces that objects present to the outside world, which helps in generating the necessary code for different programming languages.
- **IIOP (Internet Inter-ORB Protocol)** for communication

JAVA RMI

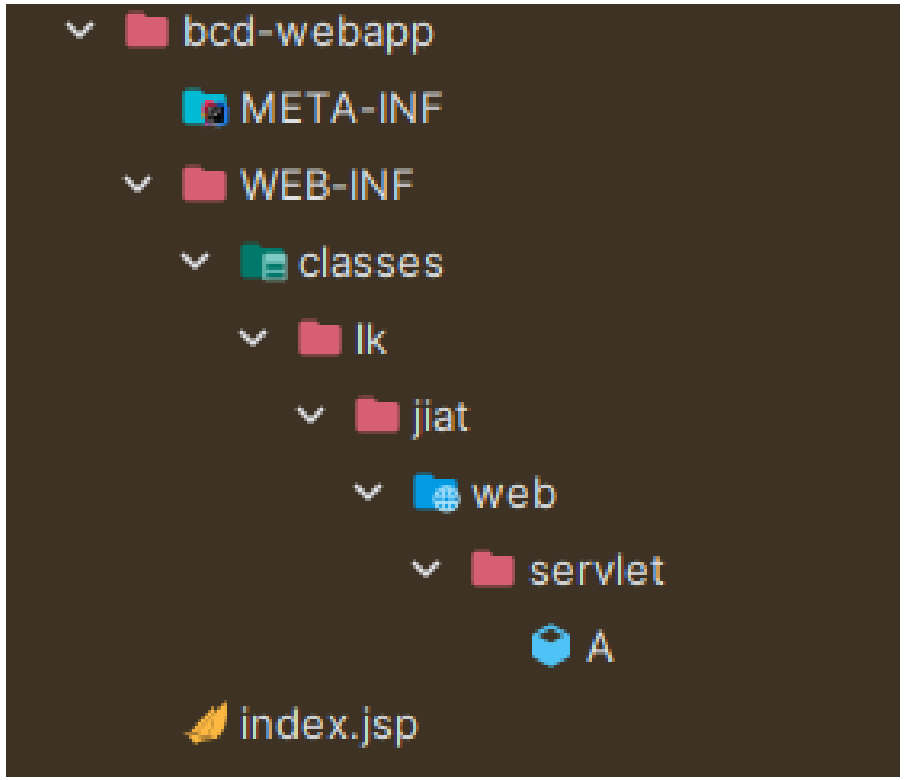
RMI stands for **Remote Method Invocation**. It is a mechanism that allows an object residing in one system (JVM) to access/invoke an object running on another JVM.

RMI is used to build distributed applications; it provides remote communication between Java programs. It is provided in the package **java.rmi**.

Architecture of an RMI Application

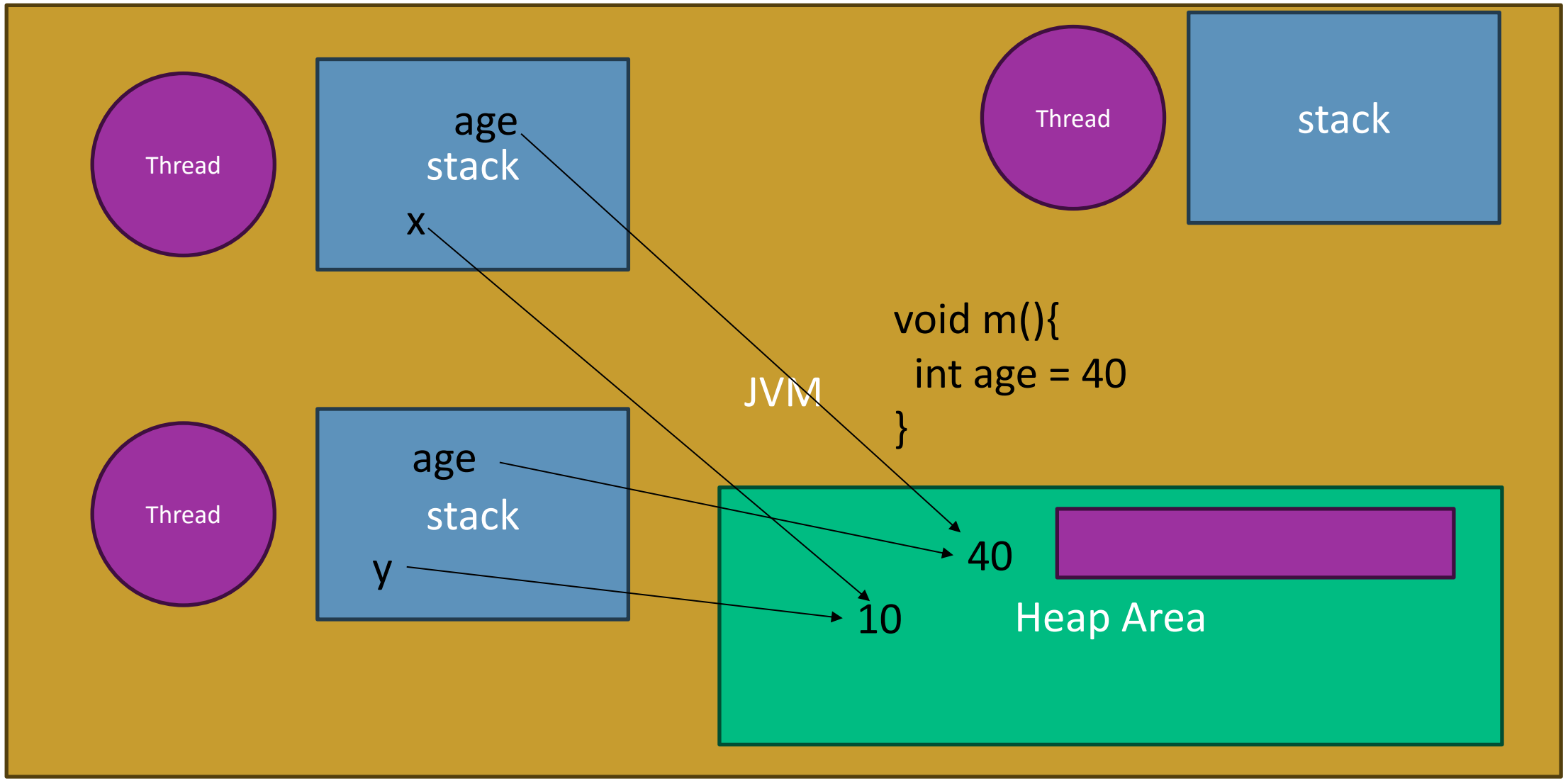


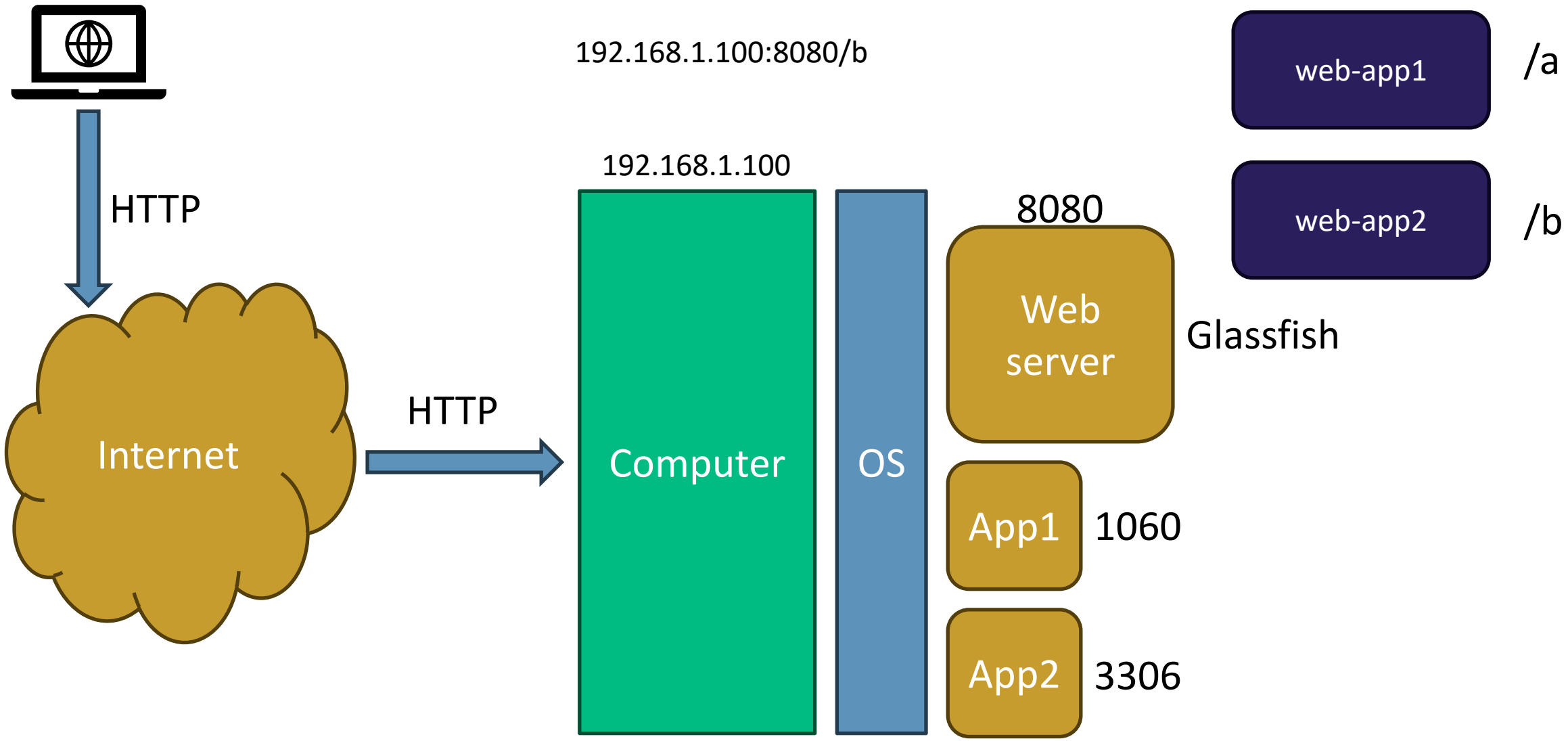
ENTERPRISE BEANS

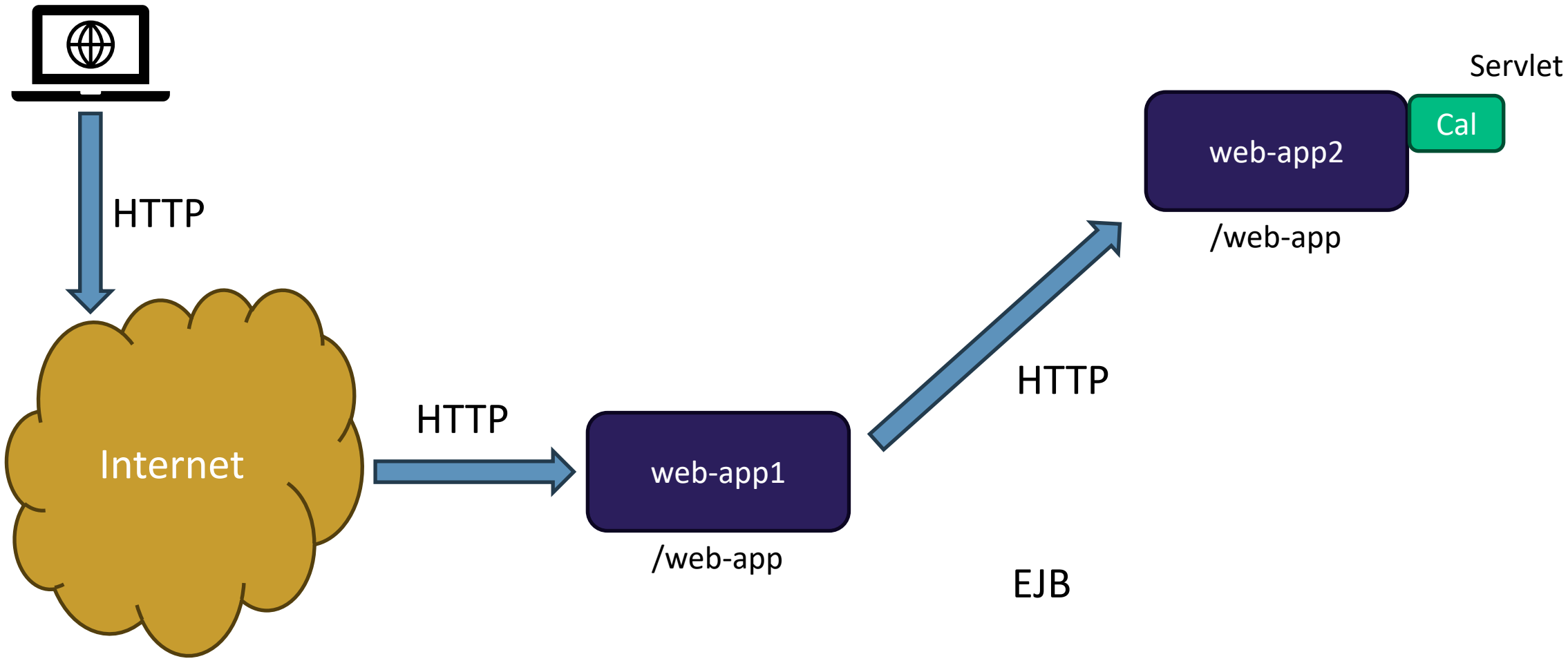



```java
public class Main {
    public static void main(String[] args) {

    }
}
```

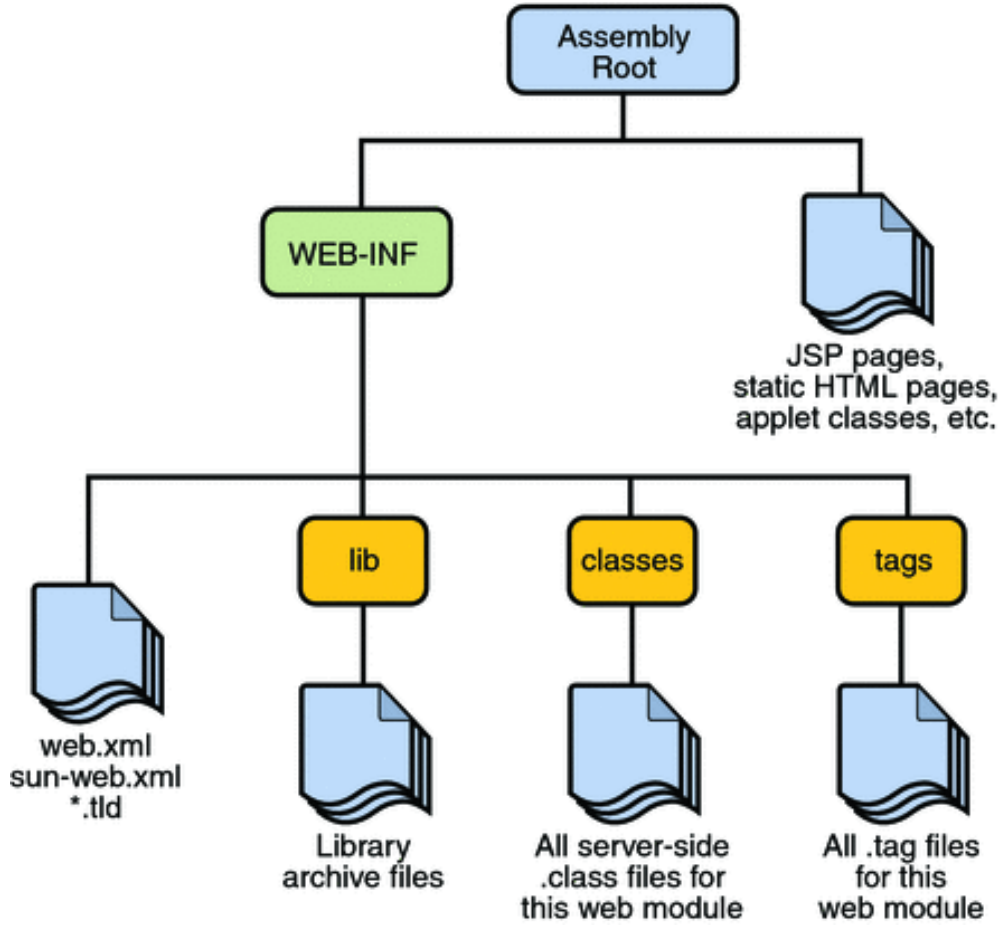





Servlet
Cal
web-app2
/web-app
HTTP
HTTP
HTTP
Internet
web-app1
/web-app
EJB

Structure of a WAR file

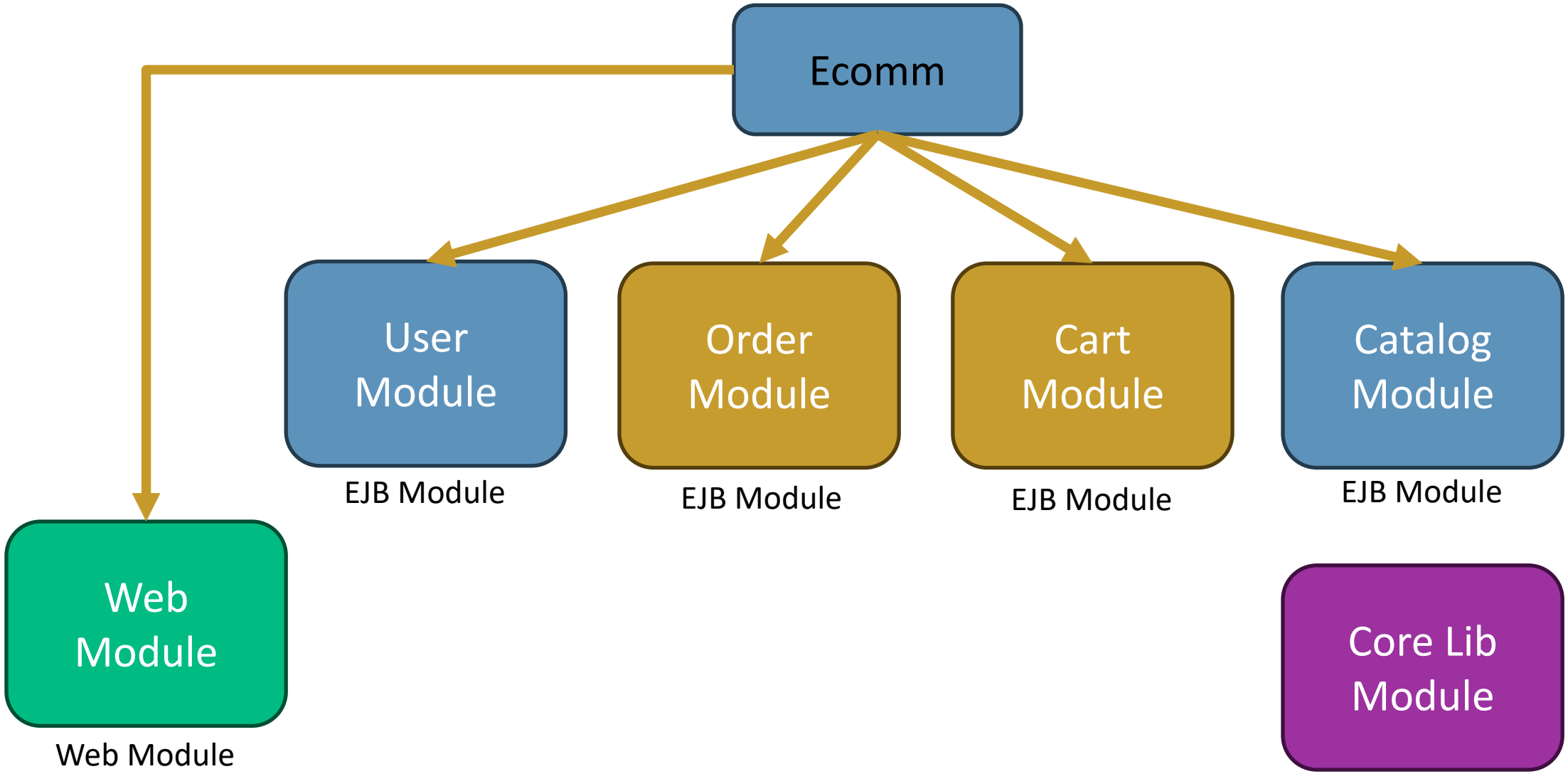
A WAR file typically stores static HTML pages, images, JavaServer pages, Java servlets, XML files, libraries, and other resources that constitute a Java web application. The top level consists of a WEB-INF directory and application-specific content such as Java server pages, static HTML pages, images, etc.

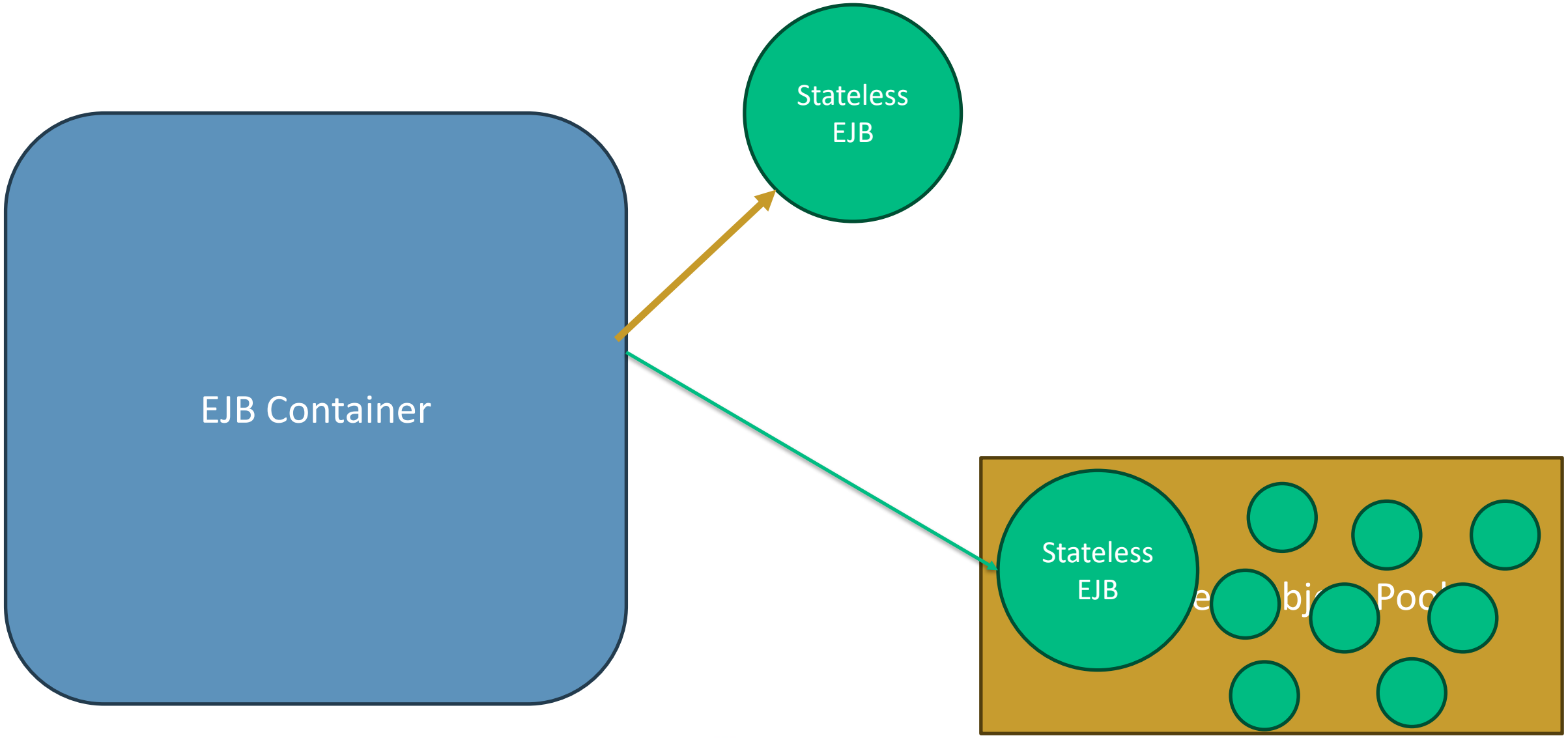


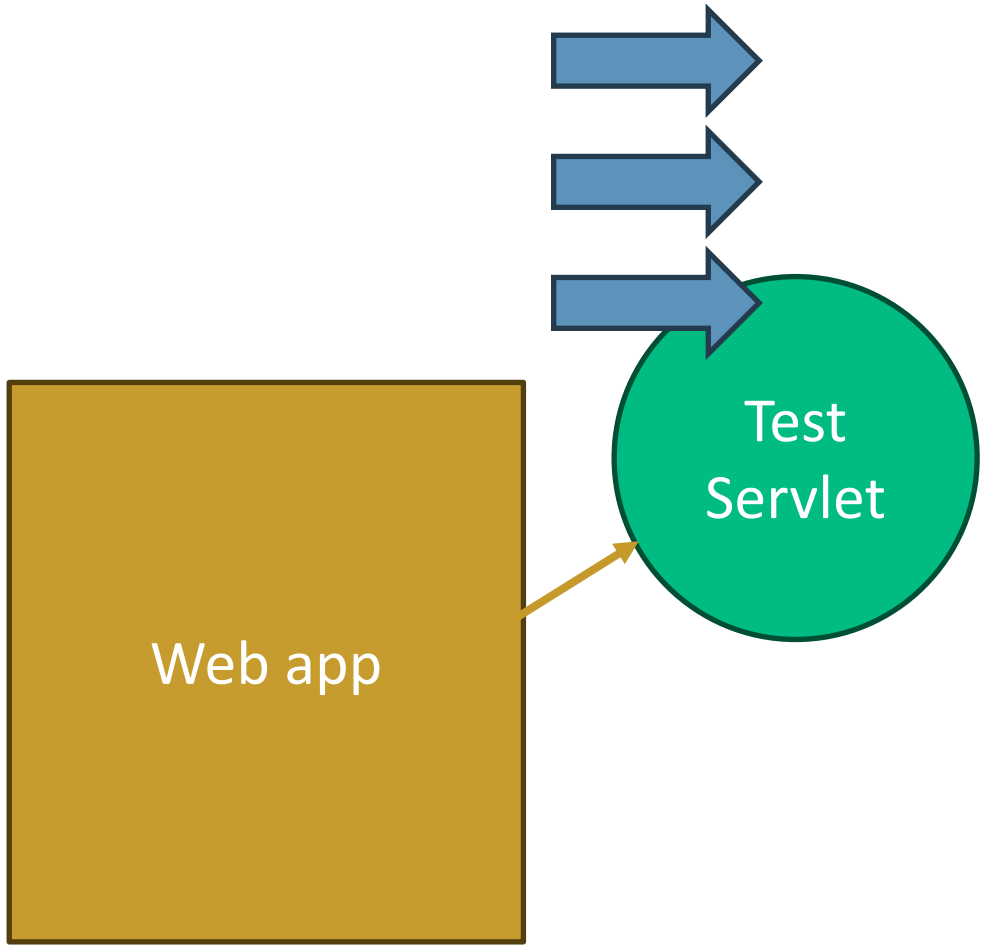
Enterprise beans are Java EE components that implement Enterprise JavaBeans (EJB) technology. Enterprise beans run in the EJB container, a runtime environment within the Application Server.

Although transparent to the application developer, the EJB container provides system-level services, such as transactions and security, to its enterprise beans. These services enable you to quickly build and deploy enterprise beans, which form the core of transactional Java EE applications.

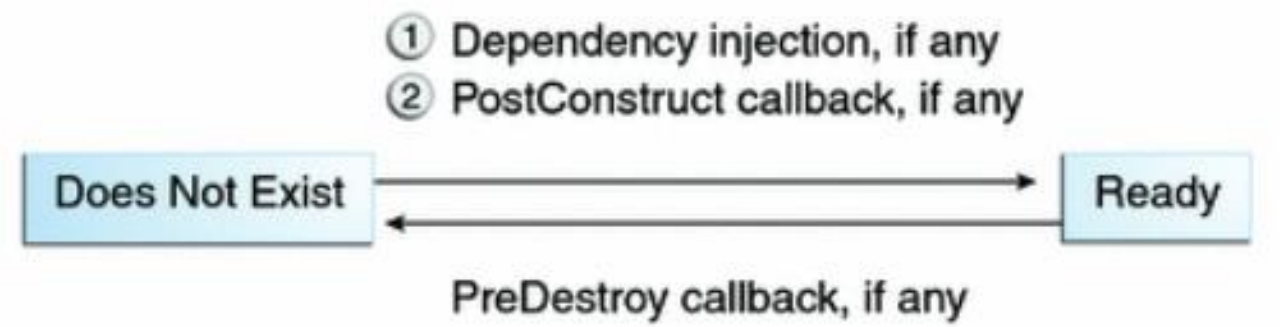
Written in the Java programming language, an enterprise bean is a server-side component that encapsulates the business logic of an application. The business logic is the code that fulfills the purpose of the application.





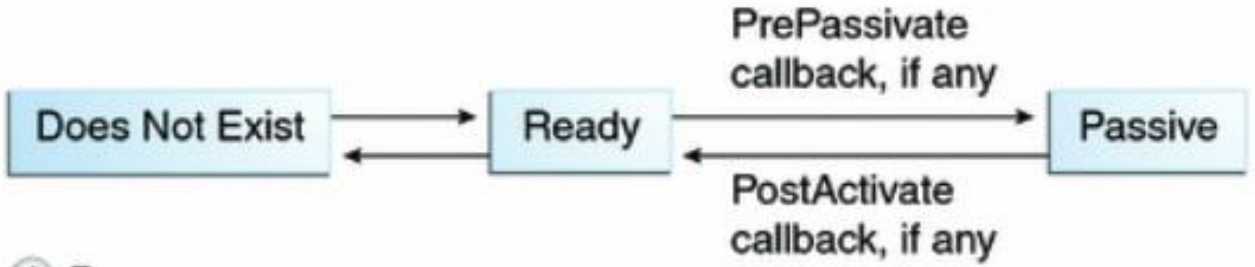

Test
Servlet
Web app

The Lifecycle of a Stateless Session Bean



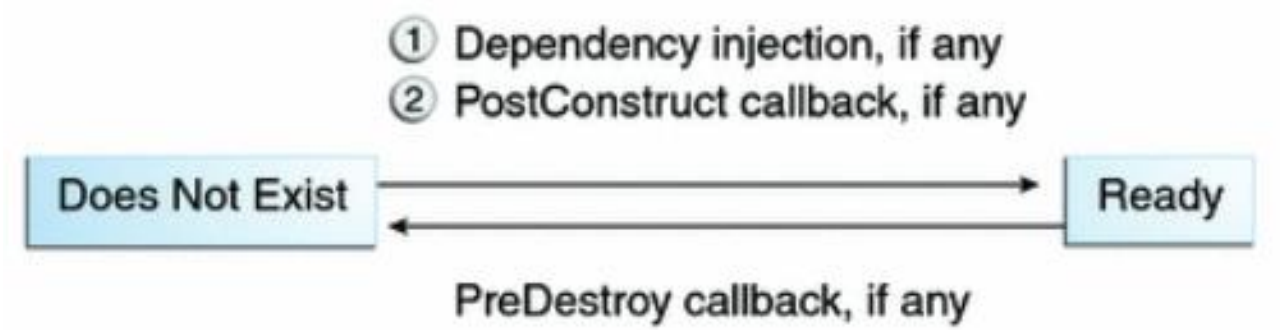
The Lifecycle of a Stateful Session Bean

① Create
② Dependency injection, if any
③ PostConstruct callback, if any
④ Init method, or ejbCreate<METHOD>, if any



① Remove
② PreDestroy callback, if any

The Lifecycle of a Singleton Session Bean



Portable JNDI Syntax

Three JNDI namespaces are used for portable JNDI lookups:

- java:global
- java:module
- java:app

java:global[/application name]/module name/enterprise bean name[/interface name]

The **java:global** JNDI namespace is the portable way of finding remote enterprise beans using JNDI lookups.

Application name and module name default to the name of the application and module minus the file extension. Application names are required only if the application is packaged within an EAR. The interface name is required only if the enterprise bean implements more than one business interface.

java:module/enterprise bean name/[interface name]

The **java:module** namespace is used to look up local enterprise beans within the same module.

The interface name is required only if the enterprise bean implements more than one business interface.

java:app[/module name]/enterprise bean name[/interface name]

The **java:app** namespace is used to look up local enterprise beans packaged within the same application. That is, the enterprise bean is packaged within an EAR file containing multiple Java EE modules.

The module name is optional. The interface name is required only if the enterprise bean implements more than one business interface..

Java Message Service

JMS

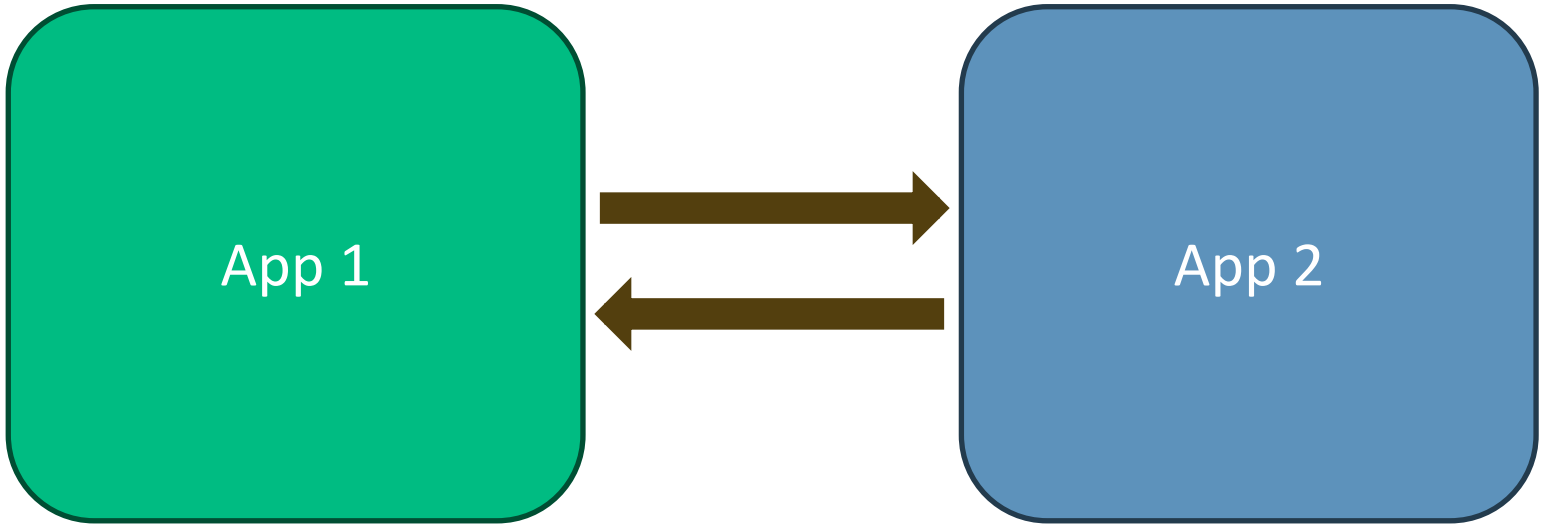
CORBA/RMI

DB

REST/SOAP

WS/Server Event

RPC



Java Message Service is an API that supports the formal communication called **messaging between computers** on a network. JMS provides a common interface for standard message protocols and message services in support of the Java programs.

JMS provides the **facility to create, send and read messages**. The JMS API reduces the concepts that a programmer must learn to use the messaging services/products and also provides the features that support the messaging applications.

JMS helps in building the communication between two or more applications in a loosely coupled manner. It means that the applications which have to communicate are not connected directly they are connected through a common destination.

Why We Need JMS?

When one application wants to send a message to another application in such a way that both applications do not know anything about each other; and even they may NOT be deployed on the same server.

JMS is also useful when we are writing any event-based application like a chat server where it needs a publish event mechanism to send messages between the server and the clients.

Note that JMS is different from RMI so there is no need for the destination object to be available online while sending a message. The publisher publishes the message and forgets it, whenever the receiver comes online, it will fetch the message. It's a very powerful solution for very common problems in today's world.

Benefits of JMS

Asynchronous

JMS is asynchronous by default. So to receive a message, the client is not required to initiate the communication. The message will arrive automatically to the client as they become available.

Reliable

JMS provides the facility of assurance that the message will be delivered once and only once. We know that duplicate messages create problems. JMS helps you avoid such problems

JMS Messaging Domains

Even before the JMS API existed, most messaging products supported either the **point-to-point** or the **publish/subscribe** approach to messaging.

The JMS also provides a separate domain for each of both approaches and defines the compliance for each domain.

Any JMS provider can implement both or one domain, it's his own choice.

The JMS provides the common interfaces which enable us to use the JMS API in such a way that it is not specific to either domain.

Point-to-Point Messaging

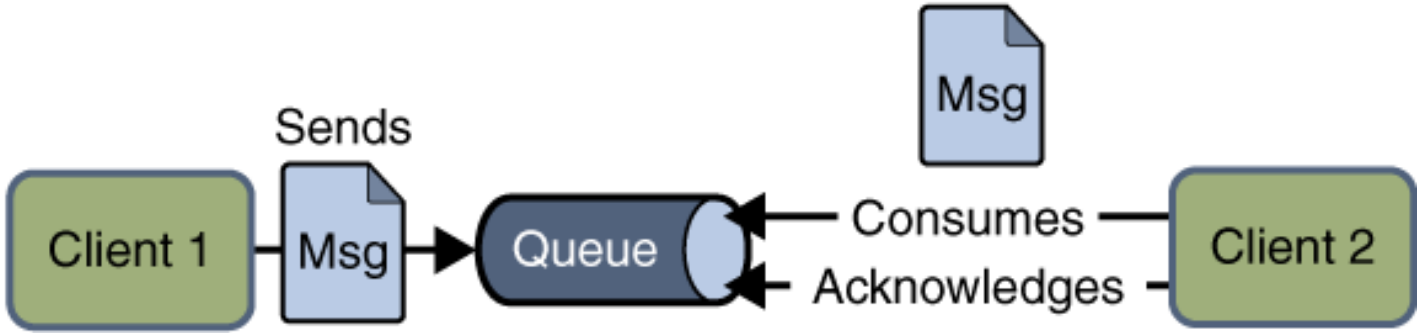
In the **point-to-point** messaging domain the application is built on the basis of message queues, senders and receivers.

Each and every message is addressed to a particular queue. Queues retain all messages sent to them until the messages are consumed or expired.

There are some characteristics of PTP messaging:

- There is only one client for each message.
- There is no timing dependency for sender and receiver of a message.
- The receiver can fetch message whether it is running or not when the sender sends the message.
- The receiver sends the acknowledgement after receiving the message.

Point-to-Point Messaging



Publish/Subscribe Messaging

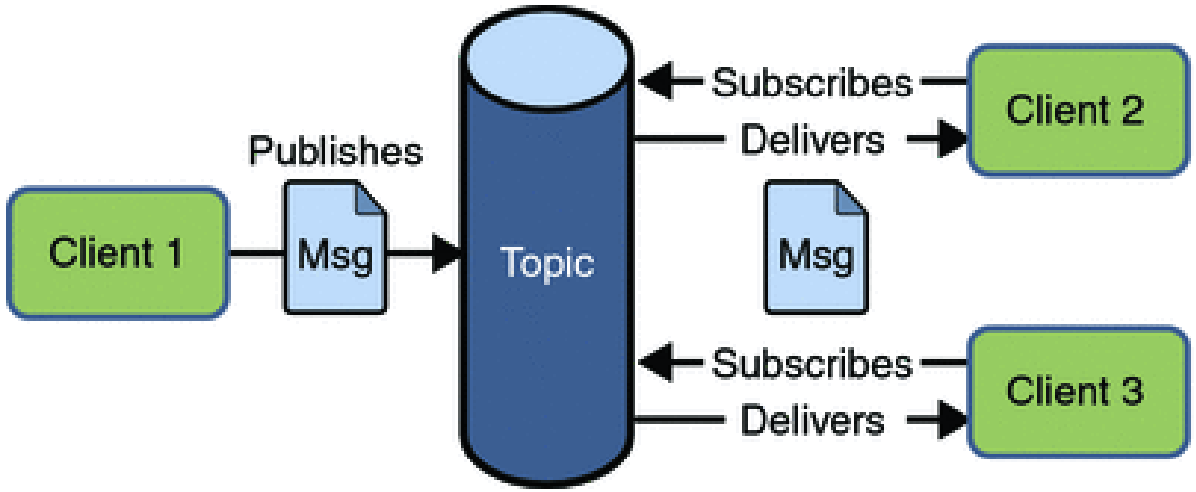
In **publish/subscribe** messaging domain, only one message is published which is delivered to all clients through **Topic** which acts as a bulletin board.

Publishers and subscribers are generally anonymous and can dynamically publish or subscribe to the topic. The Topic is responsible to hold and deliver messages. The topic retains messages as long as it takes to distribute to the present clients.

Some of the characteristics are:

- There can be multiple subscribers for a message.
- The publisher and subscribe have a timing dependency. A client that subscribes to a topic can consume only messages published after the client has created a subscription, and the subscriber must continue to be active in order for it to consume messages.

Publish/Subscribe Messaging



JMS Providers

S.N.	JMS Provider Software	Company
1	WebSphere MQ	IBM
2	Weblogic Messaging	Oracle
3	Active MQ	Apache Foundation
4	Rabbit MQ	Rabbit Technologies (obtained by Spring Source)
5	HornetQ	JBoss
6	Sonic MQ	Progress Software
7	TIBCO EMS	TIBCO
8	Open MQ	Oracle
9	SonicMQ	Aurea Software